ADX12: Direct3D 12 Userspace Driver Architecture for macOS

Microsoft_AppleDrivers - Project Genesis, Architecture, Bootstrap, Validation, and Bring-Up Specification

Independent Engineering Design Document

23 August 2026

Contents

Document Status

Working project name: `Microsoft_AppleDrivers`
Primary implementation: ADX12
Target API: Direct3D 12
Primary host: macOS on Apple Silicon
Native graphics backend: Metal
Project-owned Vulkan reference backend: AVK143
Primary external D3D12 semantic reference: vkd3d-proton
Document class: Architecture and implementation bootstrap specification
Revision date: 23 August 2026

> **Independence notice.** This is an independent engineering project proposal. It is not an official Microsoft, Apple, Khronos, Valve, CodeWeavers, Wine, or Mesa product, and the working repository name must not be presented in a way that implies sponsorship or endorsement.

This document consolidates the complete ADX12 design discussion from its initial motivation through the current repository and implementation plan. It records both the **evolution of the idea** and the **current intended architecture**, so later implementation work can distinguish historical alternatives from active design decisions.

Executive Summary

ADX12 is proposed as a **full Direct3D 12-compatible userspace graphics implementation for macOS**, rather than a narrow call-by-call D3D12-to-Metal wrapper. It should expose the Windows-facing D3D12/DXGI ABI expected by applications running through Wine or CrossOver, implement D3D12 semantics inside an ADX12-owned runtime, compile DXIL through a controlled compiler pipeline, and submit work through a native Metal backend.

The project is deliberately designed around a clean separation:

```
Application ABI
      !=
D3D12 semantic runtime
      !=
compiler representation
      !=
GPU backend
```

The native target is:

```
Windows D3D12 application
        -> Wine / CrossOver
        -> ADX12 D3D12 + DXGI ABI
        -> ADX12 runtime / device model
        -> ADXIL / NIR compiler path
        -> native Metal backend
        -> Metal
        -> AGX
```

A second path exists for validation rather than as the final fast path:

```
ADX12 workload
    -> ADX12 Vulkan reference backend
    -> Vulkan
    -> AVK143
    -> Metal
    -> AGX
```

vkd3d-proton is treated as the **primary external semantic oracle** for real-world D3D12 behavior and compatibility knowledge, but ADX12 should not inherit vkd3d-proton's Vulkan-shaped internal architecture. AVK143 replaces MoltenVK in the project design as the controlled Vulkan-over-Metal reference implementation.

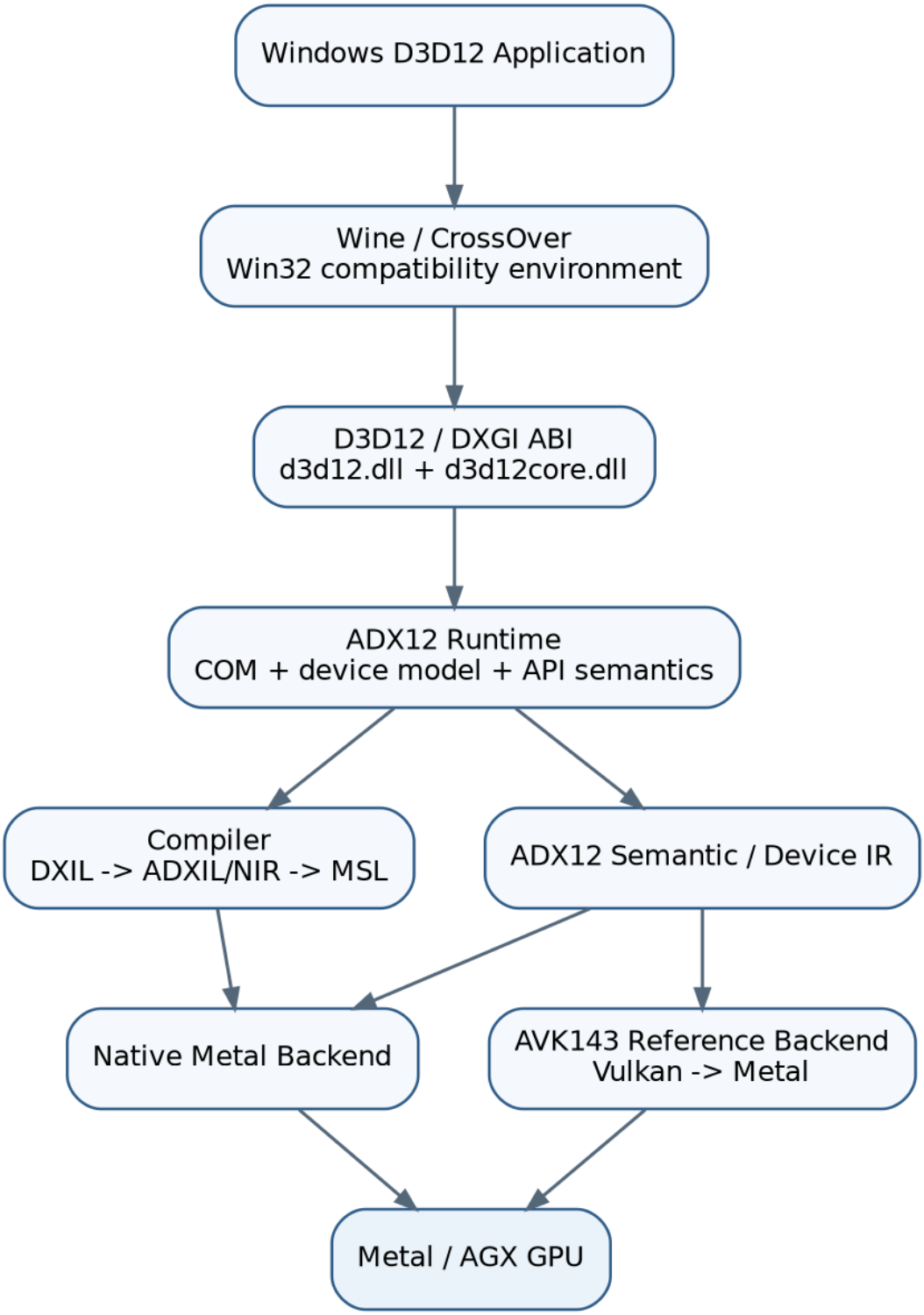


Figure 1: High-level ADX12 architecture

Project Genesis and Evolution

From D3DMetal and DXMT to ADX12

The design discussion began with a comparison between two existing macOS approaches to Windows graphics APIs:

- **D3DMetal**, Apple's Game Porting Toolkit-derived Direct3D translation technology, demonstrates that contemporary D3D11/D3D12 workloads can be translated into Metal on Apple Silicon.
- **DXMT** demonstrates an independent, open-source D3D10/D3D11 implementation aimed directly at Metal and Wine integration.

The useful lesson was not simply that another translator could be written. The larger architectural question was whether the same macOS graphics work could be organized as a **complete API implementation**, analogous in spirit to the existing AO46 and AVK143 efforts, with a stable frontend, its own runtime model, compiler pipeline, backend abstraction, and validation plan.

That question produced ADX12.

The first architectural correction: ADX12 is not WDDM

A literal Windows graphics driver uses the Windows Display Driver Model and depends on Windows runtime, user-mode driver, kernel graphics, memory-management, and scheduling contracts that macOS does not expose.

Therefore, ADX12 is **not** intended to reproduce the Windows WDDM kernel stack on macOS. The viable target is a complete **D3D12-compatible userspace implementation** that preserves application-visible semantics while using Wine/CrossOver for the surrounding Win32 environment and Metal for GPU execution.

This distinction is foundational:

```
Not the goal:
D3D12 application -> Windows WDDM UMD/KMD stack -> GPU

ADX12 goal:
D3D12 application -> Wine/CrossOver -> ADX12 userspace runtime -> Metal -> AGX
```

Evolution from "translation layer" to "owned runtime"

The design then moved away from a simplistic mapping model:

```
D3D12 call -> corresponding Metal call
```

and toward a driver-style model:

```
D3D12 API call
    -> ADX12 semantic object/state
    -> ADX12 device/runtime IR
    -> backend-specific execution
```

This is important for heaps, resource lifetime, descriptors, root signatures, command lists, queue synchronization, pipeline state, barriers, residency-like behavior, presentation, and shader compilation. These concepts do not map one-to-one onto Metal and therefore require an implementation-owned semantic layer.

vkd3d-proton becomes the main semantic oracle

The next major decision was to use the **idea and accumulated knowledge behind vkd3d-proton** without turning ADX12 into a Metal-flavored fork. vkd3d-proton already demonstrates that a replacement D3D12 implementation can expose native Win32 DLLs while targeting a different GPU API underneath. It also contains years of compatibility knowledge around command behavior, descriptors, root signatures, resource state, DXIL, pipeline state, DXR, mesh shaders, `ExecuteIndirect`, feature reporting, and application quirks.

ADX12 should study and test against that behavior, while preserving a Metal-native internal design.

AVK143 replaces MoltenVK in the validation architecture

An early bootstrap idea considered using a generic Vulkan-to-Metal implementation as a reference backend. That was superseded by a cleaner project-owned path: **AVK143**.

The current architecture therefore uses:

```
ADX12 -> Vulkan reference backend -> AVK143 -> Metal
```

rather than introducing MoltenVK as an architectural dependency.

This gives the project a controlled reference implementation that can be instrumented and debugged alongside ADX12.

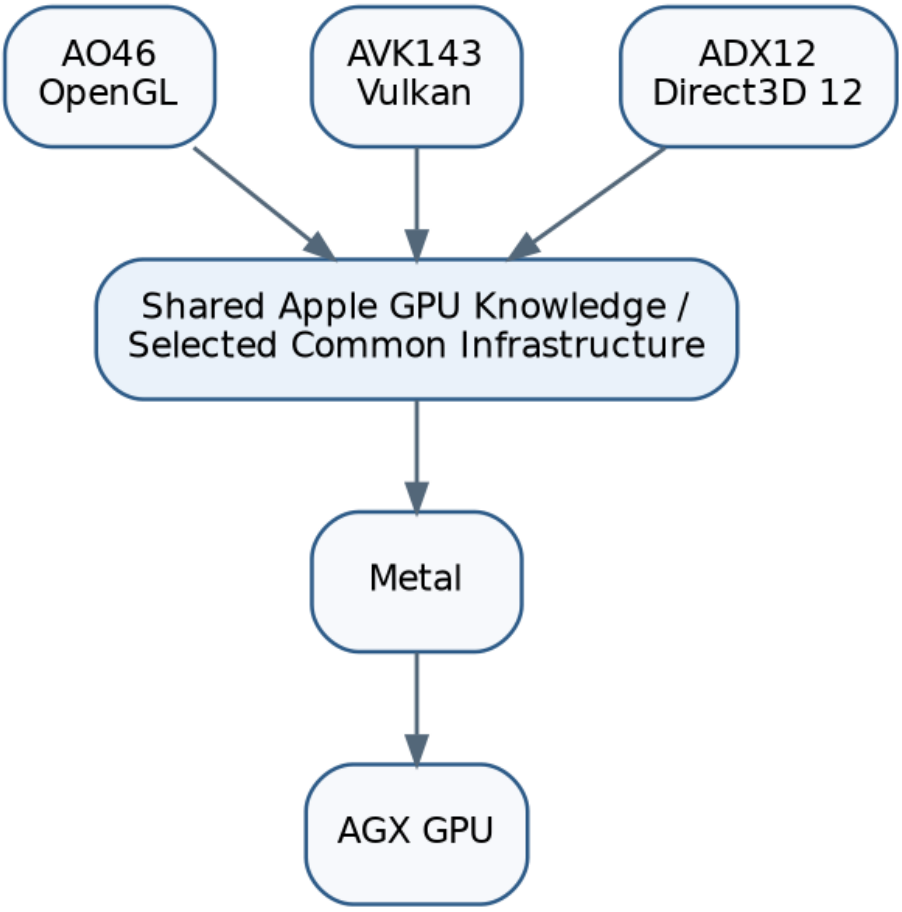


Figure 1: Driver family relationship

Consolidated Decision Record

Decision	Current position	Rationale
Project type	Full D3D12-compatible userspace implementation	A direct wrapper is too weak for D3D12 semantics
Win32 environment	Wine / CrossOver	Avoid reimplementing Windows itself
Native backend	Metal	Direct macOS/Apple Silicon target
Reference Vulkan backend	AVK143	Project-controlled Vulkan-over-Metal validation path
Main D3D12 semantic oracle	vkd3d-proton	Mature real-world D3D12 compatibility knowledge
Microsoft architecture references	DirectX-Headers, D3D12TranslationLayer, D3D11On12, OpenCLOn12, DirectX-Specs	Official ABI and implementation-side design material
Bootstrap DXIL route	DXIL -> dxil-spirv -> SPIR-V -> Mesa NIR	Gets runtime development moving before native DXIL frontend exists
Long-term shader route	DXIL -> ADXIL -> NIR -> MSL	Removes the temporary SPIR-V bridge
Common shader IR	Mesa NIR	Shared optimization and lowering infrastructure
Metal C++ interface	metal-cpp	Low-overhead, header-only C++ Metal interface
DXGI	Separate ADX12 subsystem	Presentation and adapter behavior should not pollute core runtime
Licensing strategy	Keep permissive core clean; use LGPL projects primarily as references unless deliberately integrated under their terms	Avoid accidental license contamination

Reading Guide

The rest of the document is organized as follows:

1. project objective, goals, and non-goals;
2. API/runtime/compiler/backend architecture;
3. D3D12 object and execution models;
4. compiler and shader strategy;
5. Metal and AVK143 backends;
6. reference projects and dependency policy;
7. validation and differential testing;
8. repository organization;
9. phased bring-up roadmap;
10. licensing, risks, contribution rules, and long-term expansion.

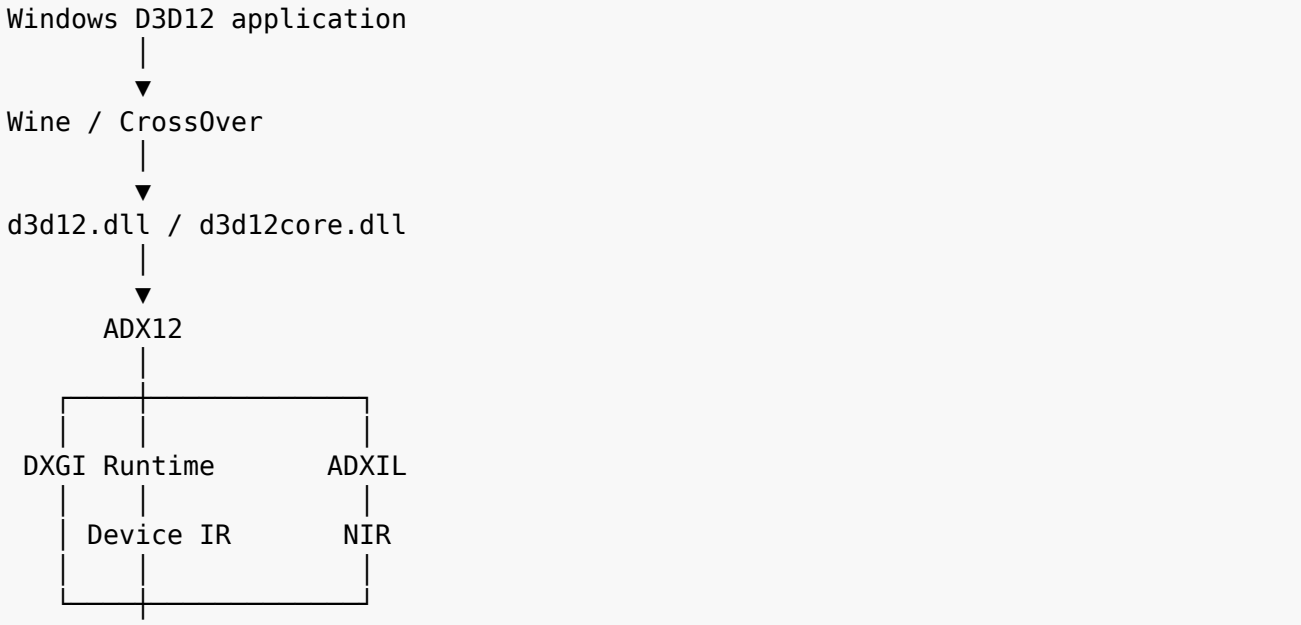
1. Project Objective

ADX12 is intended to become a **full Direct3D 12-compatible userspace graphics implementation for macOS**, designed with the same general philosophy as AO46 and AVK143:

- expose the API and ABI expected by applications;
- implement the API semantics inside a dedicated driver/runtime;
- maintain an internal device and resource model;
- compile shaders through a controlled compiler stack;
- drive Apple GPUs through a native Metal backend;
- keep platform integration, runtime semantics, compiler logic, and backend code clearly separated.

ADX12 is **not** intended to be only a thin D3D12-to-Metal translation DLL.

The long-term model is:



```
      |
      ▼
Metal Backend
      |
      ▼
  metal-cpp
      |
      ▼
    Metal
      |
      ▼
    AGX
```

The core design goal is:

> **Implement D3D12-visible behavior faithfully while designing the internal runtime, compiler, memory model, synchronization model, and GPU backend specifically around macOS, Apple Silicon, and Metal.**

---

2. Design Philosophy

ADX12 should follow the rule:

```
Frontend ≠ Runtime ≠ Compiler ≠ Backend
```

A Windows application should see normal Direct3D interfaces:

```
D3D12CreateDevice(...)
ID3D12Device
ID3D12CommandQueue
ID3D12GraphicsCommandList
ID3D12Resource
ID3D12DescriptorHeap
ID3D12PipelineState
IDXGISwapChain
```

Internally, ADX12 should convert these into its own semantic objects rather than mapping calls directly to Metal.

Example:

```
ID3D12Resource
       │
       ▼
  ADXResource
       │
       ├── ADXAllocation
       └── ADXResourceState
       │
       ▼
ADXMetalResource
       │
       ▼
MTLBuffer / MTLTexture
```

Descriptors should similarly pass through an ADX12-owned model:

```
D3D12 descriptor
       │
       ▼
 ADXDescriptor
       │
       ▼
ADXBindingLayout
       │
       ▼
Metal argument-buffer / resource binding
```

This prevents the entire project from becoming permanently shaped around another graphics API.

3. Goals

ADX12 should eventually provide:

- D3D12-compatible Win32-facing DLL entry points.
- `d3d12.dll` / `d3d12core.dll` compatibility.
- DXGI integration.
- D3D12 COM object behavior.
- Device and capability enumeration.
- Graphics, compute, and copy queues.
- Command allocators and command lists.
- Resource and heap management.
- Descriptor heaps and root signatures.
- Pipeline State Objects.
- Queries and fences.
- Resource-state tracking.
- Barriers and synchronization.
- DXIL ingestion.
- NIR-based shader compilation.
- Native MSL generation.
- Native Metal command submission.
- CAMetalLayer-based presentation.
- Apple-Silicon-aware memory behavior.
- Differential validation using AVK143 and vkd3d-proton.
- Future support for advanced D3D12 features such as DXR, mesh shaders, enhanced barriers, and `ExecuteIndirect`.

---

4. Non-Goals

The project should **not** begin as:

- a fork of vkd3d-proton with Vulkan calls replaced by Metal;
- a D3DMetal clone;
- a DXMT-for-D3D12 rewrite;
- a Windows WDDM kernel-mode driver;
- a full Win32 compatibility environment;
- a monolithic `translate.cpp`;
- a direct one-D3D12-call → one-Metal-call shim.

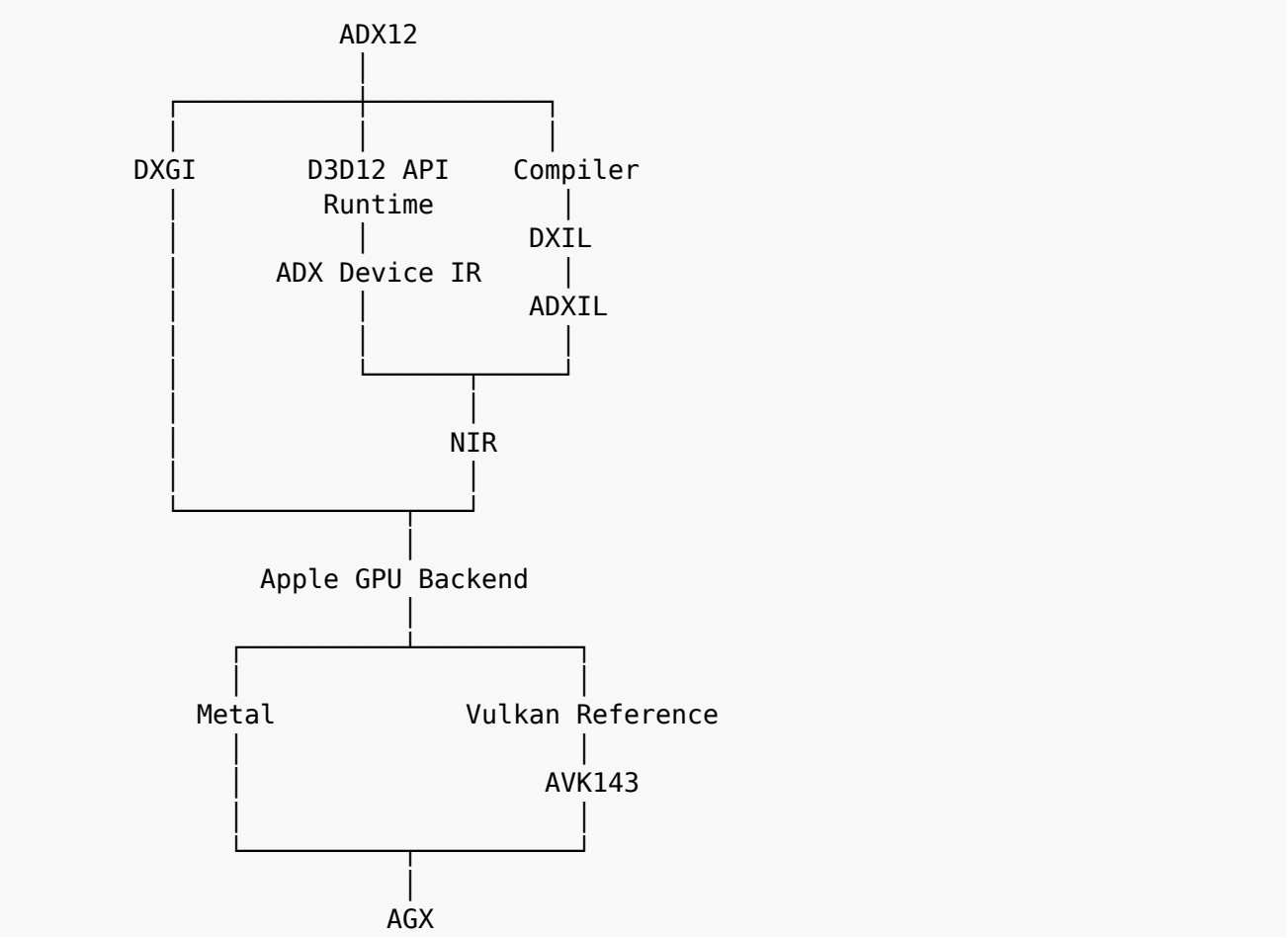
Wine or CrossOver should provide the Windows compatibility layer.

ADX12 should own the graphics implementation.

```
Windows application
        │
        ▼
Wine / CrossOver
        │
        ├── PE loading
        ├── ntdll
        ├── kernel32
        ├── user32
        ├── threading
        ├── filesystem
        └── Win32 windowing
        │
        ▼
ADX12
        │
        ├── D3D12 ABI
        ├── DXGI
        ├── DXIL
        ├── resources / heaps
        ├── descriptors
        ├── command lists
        ├── queues
        ├── pipelines
        ├── synchronization
        └── presentation
        │
        ▼
Metal
```

---

5. High-Level Architecture



The Vulkan path exists primarily for **bring-up and differential validation**. The final intended fast path is:

```
D3D12
  ↓
ADX12
  ↓
Metal
  ↓
AGX
```

---

6. Core Runtime Objects

ADX12 should maintain its own internal objects.

Initial object family:

```
ADXDevice
ADXQueue
ADXCommandAllocator
ADXCommandStream
ADXCommandList
ADXResource
ADXAllocation
ADXHeap
ADXDescriptor
ADXDescriptorHeap
ADXBindingLayout
ADXRootSignature
ADXPipeline
ADXPipelineCache
ADXBarrierGraph
ADXFence
ADXQuery
ADXSurface
```

These objects should represent D3D12-visible semantics independently from any backend API.

---

7. External Resource Strategy

External projects should be divided into four groups:

1. **Core build dependencies**
2. **Toolchain dependencies**
3. **Reference implementations**
4. **Tests and debugging resources**

Do not treat every useful upstream repository as a permanent dependency.

---

8. Core Dependencies

8.1 Microsoft DirectX-Headers

Repository:

- https://github.com/microsoft/DirectX-Headers

Use for:

- official D3D12 headers;
- interface definitions;
- IDLs;
- DXGI formats;
- GUIDs;
- d3dx12 helpers.

Classification:

```
Mandatory ABI dependency
```

8.2 DirectX Shader Compiler (DXC)

Repository:

- https://github.com/microsoft/DirectXShaderCompiler

Use for:

- HLSL → DXIL;
- DXIL container reference;
- DXIL assembler/disassembler;
- validator infrastructure;
- Shader Model behavior and documentation.

Classification:

```
Compiler/toolchain dependency
```

8.3 dxil-spirv

Repository:

- https://github.com/HansKristian-Work/dxil-spirv

Use for:

- initial Shader Model 6 DXIL parsing;
- DXIL → SPIR-V;
- shader regression tests;
- bootstrap path into Mesa NIR.

Classification:

```
Core bootstrap compiler dependency
```

This avoids making the first driver milestone depend on writing a full native DXIL frontend.

8.4 Mesa

Repository:

- https://gitlab.freedesktop.org/mesa/mesa

Use for:

- NIR;
- `spirv_to_nir`;
- optimization passes;
- lowering infrastructure;
- shared compiler utilities;
- Microsoft/DXIL-related compiler research;
- integration with existing KosmicKrisp/NIR work.

Classification:

```
Core compiler infrastructure
```

8.5 Apple metal-cpp

Repository:

- https://github.com/apple/metal-cpp

Use for:

- native C++17 Metal integration;
- device/resource/queue/command access;
- minimizing Objective-C++ surface area.

Classification:

```
Core Metal backend interface
```

8.6 SPIRV-Cross

Repository:

- https://github.com/KhronosGroup/SPIRV-Cross

Use for:

- MSL comparison;
- binding validation;
- reference shader translation;
- bootstrap debugging.

Classification:

```
Compiler/reference utility
```

8.7 SPIRV-Tools

Repository:

- https://github.com/KhronosGroup/SPIRV-Tools

Use for:

- SPIR-V validation;
- disassembly;
- optimization;
- debugging the temporary DXIL → SPIR-V path.

8.8 SPIRV-Headers

Repository:

- https://github.com/KhronosGroup/SPIRV-Headers

Use for:

- shared SPIR-V definitions required by compiler components.

8.9 Wine

Repository:

- https://gitlab.winehq.org/wine/wine

Use for:

- Win32 ABI behavior reference;
- COM reference;
- PE/DLL integration;
- `widl`;
- Wine-facing driver integration.

Classification:

```
Platform integration reference/tooling
```

Wine should remain responsible for the Windows compatibility environment.

---

9. Primary D3D12 Reference Implementation: vkd3d-proton

Repository:

- https://github.com/HansKristian-Work/vkd3d-proton

vkd3d-proton should be the **primary external semantic reference** for ADX12.

It should be studied for:

- D3D12 ABI behavior;
- COM lifetime and identity semantics;
- device creation;
- feature reporting;
- command queues;
- command allocators;
- command lists;
- resources and heaps;
- descriptor heaps;
- root signatures;
- Pipeline State Objects;
- barriers;
- synchronization;
- DXIL handling;
- shared resources;
- DXR;
- mesh shaders;
- `ExecuteIndirect`;
- application compatibility quirks;
- Agility SDK behavior.

Its role is:

```
vkd3d-proton
     │
     ├── D3D12 ABI/reference behavior
     ├── COM semantics
     ├── resource semantics
     ├── descriptor semantics
     ├── synchronization semantics
     ├── shader behavior
     ├── feature reporting
     └── compatibility knowledge
               │
               ▼
            ADX12
               │
               ▼
      ADX12-owned Metal design
```

9.1 What Should Be Reused

Reuse:

```
knowledge
semantics
behavioral expectations
test ideas
architecture lessons
edge-case discoveries
```

Do **not** automatically inherit:

```
Vulkan-shaped internal objects
Vulkan descriptor architecture
Vulkan synchronization architecture
VkBuffer/VkImage-centric resource representation
```

vkd3d-proton conceptually follows:

```
D3D12
  ↓
vkd3d-proton
  ↓
Vulkan
```

ADX12 should follow:

```
D3D12
  ↓
ADX12 API layer
  ↓
ADX12 semantic/device IR
  ↓
Metal backend
  ↓
Metal
```

9.2 Repository Placement

Preferred:

```
references/
    refs.lock
    fetch-references.sh
```

with a pinned vkd3d-proton revision.

Alternative:

```
external/reference/vkd3d-proton/
```

9.3 Reference Notes

Create ADX12-side notes such as:

```
docs/reference/
├── VKD3DDescriptors.md
├── VKD3DEnhancedBarriers.md
├── VKD3DResourceModel.md
├── VKD3DCommandSubmission.md
└── VKD3DFeatureQueries.md
```

Each should compare:

```
Microsoft specification
        +
vkd3d-proton implementation
        +
Microsoft translation-layer/reference behavior
        ↓
ADX12 implementation
        +
Metal-specific design
```

---

10. Other Important Reference Projects

10.1 Upstream Wine VKD3D

Repository:

- https://gitlab.winehq.org/wine/vkd3d

Use for:

- general-purpose D3D12 behavior;
- cleaner upstream comparisons;
- API semantics independent of Proton-specific optimizations.

10.2 DXMT

Repository:

- https://github.com/3Shain/dxmt

Use for:

- Wine ↔ macOS ↔ Metal integration;
- DXGI design;
- command submission;
- Metal-specific implementation strategies;
- shader/backend architecture.

DXMT is a D3D10/11 implementation, not D3D12, but its platform architecture is highly relevant.

10.3 DXVK

Repository:

- https://github.com/doitsujin/dxvk

Use for:

- DXGI;
- COM behavior;
- DLL deployment;
- logging;
- caches;
- adapter selection;
- Wine integration;

- frontend/runtime/platform separation.

---

11. AVK143 Replaces MoltenVK in the Architecture

ADX12 should **not depend on MoltenVK** for the project design.

AVK143 should instead be the project-owned Vulkan-over-Metal reference backend.

This produces a cleaner internal family:

```
Khronos_AppleICDs
├── AO46
└── AVK143

Microsoft_AppleDrivers
└── ADX12
```

For ADX12 validation:

```
ADX12 Vulkan reference backend
          │
          ▼
        AVK143
          │
          ▼
        Metal
          │
          ▼
         AGX
```

Advantages:

- the Vulkan reference path is controlled by the same project family;
- no dependence on another Vulkan→Metal implementation;
- AVK143 can be instrumented specifically for ADX12 validation;
- bugs can be traced across both sides;
- feature exposure can be coordinated;
- the same Apple GPU backend knowledge can be reused;
- differential testing becomes more meaningful.

MoltenVK may still be read historically as an external implementation if useful, but it is **not part of the required ADX12 bootstrap stack**.

---

12. Microsoft Open-Source Driver-Side References

12.1 D3D12TranslationLayer

Repository:

- https://github.com/microsoft/D3D12TranslationLayer

Study for:

- descriptor binding;
- resource management;
- memory management;
- batching;
- PSO compilation;
- residency concepts.

12.2 D3D11On12

Repository:

- https://github.com/microsoft/D3D11On12

Study for:

- Microsoft user-mode graphics-driver concepts;
- DDI-style implementation;
- resource and command translation;
- higher-level API implementation over an explicit lower-level graphics API.

12.3 OpenCLOn12

Repository:

- https://github.com/microsoft/OpenCLOn12

Study because it demonstrates Microsoft using Mesa compiler infrastructure inside a Microsoft API compatibility project.

This is conceptually valuable for ADX12 because ADX12 also intends to combine Microsoft API semantics with Mesa compiler infrastructure.

12.4 DirectX-Specs

Repository:

- https://github.com/microsoft/DirectX-Specs

Use for:

- enhanced barriers;
- Shader Models;
- DXR;
- mesh shaders;
- sampler feedback;
- Feature Level 12_2;
- GPU upload heaps;
- newer D3D12 features.

---

13. Testing and Debugging Resources

13.1 DirectX Graphics Samples

Repository:

- https://github.com/microsoft/DirectX-Graphics-Samples

Use as known-good workloads for:

- basic rendering;
- `ExecuteIndirect`;
- multi-engine workloads;
- mesh shaders;
- VRS;
- DXR.

13.2 D3D12 Memory Allocator

Repository:

- https://github.com/GPUOpen-LibrariesAndSDKs/D3D12MemoryAllocator

Use as a reference for:

- heap allocation;
- suballocation;
- placed resources;
- aliasing;
- heap tiers;
- memory budgets.

13.3 PixEvents

Repository:

- https://github.com/microsoft/PixEvents

Use for:

- `BeginEvent`;
- `EndEvent`;
- markers;
- debugging-event compatibility.

13.4 RenderDoc

Repository:

- https://github.com/baldurk/renderdoc

Use for:

- D3D12 capture/replay concepts;
- resource inspection;
- shader inspection;
- DXIL handling;
- debugging-model reference.

13.5 DirectXTK12 / DirectXTex / DirectXMath

Repositories:

- https://github.com/microsoft/DirectXTK12
- https://github.com/microsoft/DirectXTex
- https://github.com/microsoft/DirectXMath

Use mainly for:

- test applications;
- DDS/BC texture validation;
- sample workloads;
- utility code in tests.

---

14. Shader Compiler Strategy

The compiler should be developed in two stages.

14.1 Stage 1: Bootstrap Shader Path

Initial path:

```
HLSL
 │
 ▼
DXC
 │
 ▼
DXIL
 │
 ▼
dxil-spirv
 │
 ▼
SPIR-V
 │
 ▼
Mesa spirv_to_nir
 │
 ▼
NIR
 │
 ▼
KosmicKrisp / shared Apple lowering
 │
 ▼
MSL
 │
 ▼
Metal
```

Advantages:

- runtime development can begin immediately;
- existing DXIL parsing work is reused;
- SPIR-V can be validated;
- Mesa already has a mature SPIR-V → NIR path;
- shader bugs can be isolated from runtime bugs.

14.2 Stage 2: ADXIL Native Frontend

After the runtime works, build:

```
ADX12/compiler/ADXIL/
```

Long-term path:

```
DXIL
 │
 ▼
DXIL container parser
 │
 ▼
LLVM/DXIL instruction decoder
 │
 ▼
ADXIL semantic IR
 │
 ▼
NIR
 │
 ▼
shared Apple lowering
 │
 ▼
MSL
```

The final goal is:

```
DXIL → ADXIL → NIR
```

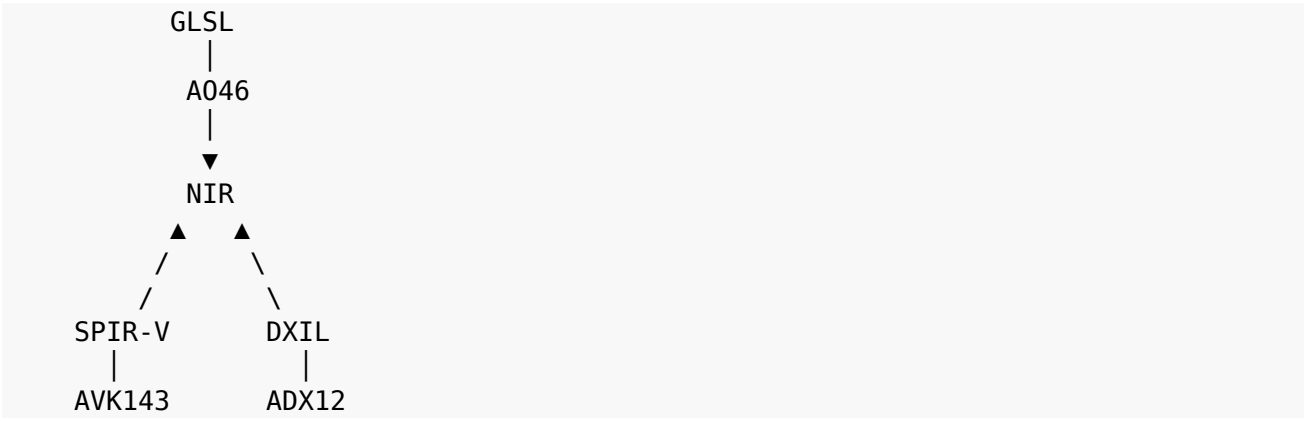
replacing:

```
DXIL → dxil-spirv → SPIR-V → NIR
```

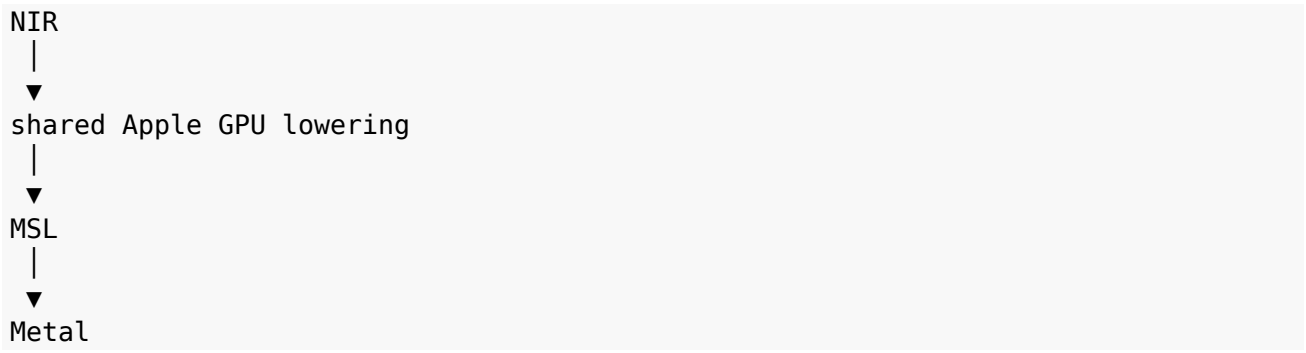
---

15. Common Compiler Geometry

The long-term compiler family can become:



Below NIR:



This permits common optimization and Apple-specific lowering work where appropriate.

---

16. Resource and Memory Model

D3D12 memory concepts such as:

```
D3D12_HEAP_TYPE_DEFAULT
D3D12_HEAP_TYPE_UPLOAD
D3D12_HEAP_TYPE_READBACK
```

should first map into an ADX12-owned memory model.

```
D3D12 heap/resource
        │
        ▼
   ADXHeap
        │
        ▼
ADXMemoryAllocation
        │
        ▼
Metal resource allocation
        │
        ├── MTLHeap
        ├── MTLBuffer
        └── MTLTexture
```

The implementation should preserve D3D12-visible behavior without pretending Apple Silicon has a discrete-Windows-GPU memory topology.

Apple unified memory should be treated as an implementation opportunity, not as an excuse to violate D3D12 semantics.

---

17. Command Submission Model

D3D12 exposes:

```
CommandAllocator
       |
       ▼
GraphicsCommandList
       |
       ▼
CommandQueue
```

ADX12 should introduce:

```
ADXCommandAllocator
       |
       ▼
ADXCommandStream
       |
       ▼
ADXCommandQueue
       |
       ▼
Backend
```

Potential internal commands:

```
ADX_CMD_DRAW
ADX_CMD_DRAW_INDEXED
ADX_CMD_DISPATCH
ADX_CMD_COPY_BUFFER
ADX_CMD_COPY_TEXTURE
ADX_CMD_BARRIER
ADX_CMD_EXECUTE_INDIRECT
```

The Metal backend consumes these commands and creates appropriate Metal command encoders and command buffers.

---

18. Synchronization and Barrier Model

Suggested subsystem:

```
ADX12/sync/
├── ADXBarrier.*
├── ADXFence.*
├── ADXEvent.*
├── ADXResourceState.*
└── ADXQueueSync.*
```

Conceptually:

```
D3D12_RESOURCE_BARRIER
          │
          ▼
   ADXBarrierGraph
          │
          ▼
dependency analysis
          │
          ▼
Metal barriers / fences / events
```

Do not assume every D3D12 barrier maps cleanly to a single Metal primitive.

---

19. Descriptor and Root-Signature Model

Suggested subsystem:

```
ADX12/descriptor/
├── ADXRootSignature.*
├── ADXDescriptorHeap.*
├── ADXDescriptorTable.*
├── ADXSamplerHeap.*
└── ADXBindless.*
```

Conceptually:

```
D3D12 Root Signature
        │
        ▼
 ADX Binding Layout
        │
        ▼
NIR resource bindings
        │
        ▼
Metal argument buffers
        │
        ▼
Metal resources
```

---

20. DXGI and Presentation

Keep DXGI separate from the core runtime.

Objects:

```
ADXGIFactory
ADXGIAdapter
ADXGIOutput
ADXGISwapChain
ADXGIFormat
```

Presentation path:

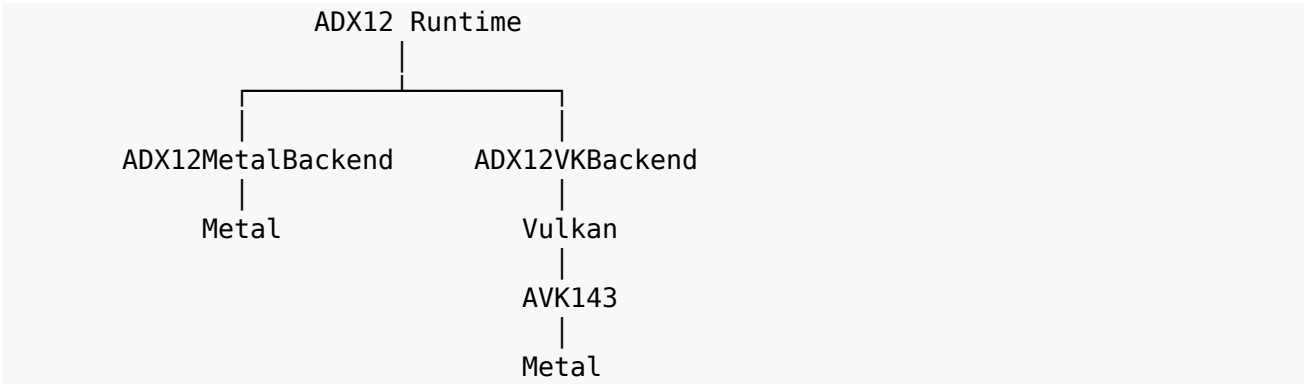
```
IDXGISwapChain
      |
      ▼
ADXGISwapChain
      |
      ▼
 ADXSurface
      |
      ▼
CAMetalLayer
      |
      ▼
Metal drawable
```

Wine/CrossOver supplies the Win32 window environment around the swapchain.

---

21. Dual-Backend Bring-Up Strategy

Initial architecture:

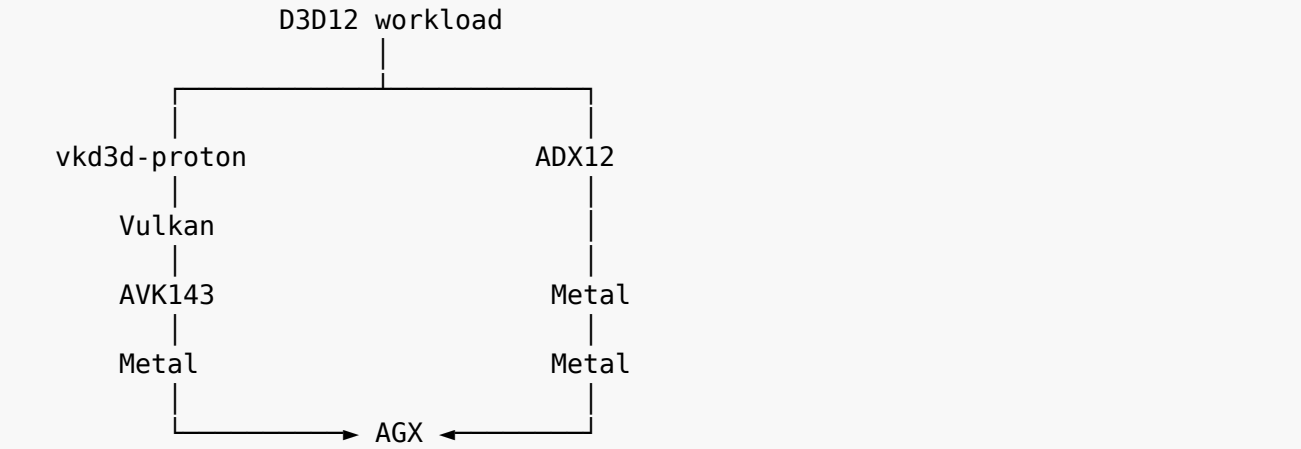


The Vulkan backend is a **validation backend**, not the final architecture.

---

22. Differential Validation

Use the same D3D12 workload through multiple paths:



Compare:

- rendered output;
- buffer contents;
- texture contents;
- descriptor behavior;
- barrier behavior;
- query results;
- compute output;
- feature reporting.

Bring-up sequence:

```
device creation
  ↓
triangle
  ↓
buffers
  ↓
textures
  ↓
descriptor heaps
  ↓
root signatures
  ↓
barriers
  ↓
compute
  ↓
multiple queues
  ↓
advanced D3D12 features
```

Interpretation:

```
vkd3d-proton → AVK143 = correct
ADX12 → Metal         = incorrect
```

likely indicates an ADX12 runtime or Metal mapping problem.

If both fail similarly, inspect AVK143/shared Apple-side infrastructure or a lower-level API/hardware limitation.

---

23. Proposed Repository Layout

```
Microsoft_AppleDrivers/
│
├── README.md
├── LICENSE
├── THIRD_PARTY.md
│
├── docs/
│   ├── ADX12Architecture.md
│   ├── D3D12MetalMapping.md
│   ├── DXILNIRMapping.md
│   ├── DXGIModel.md
│   ├── MemoryModel.md
│   ├── DescriptorModel.md
│   ├── Synchronization.md
│   ├── FeatureMatrix.md
│   ├── REFERENCES.md
│   └── reference/
│       ├── VKD3DDescriptors.md
│       ├── VKD3DEnhancedBarriers.md
│       ├── VKD3DResourceModel.md
│       └── VKD3DCommandSubmission.md
│
├── ADX12/
│   │
│   ├── api/
│   │   ├── device/
│   │   ├── command/
│   │   ├── resource/
│   │   ├── heap/
│   │   ├── descriptor/
│   │   ├── pipeline/
│   │   ├── query/
│   │   └── sync/
│   │
│   ├── runtime/
│   │   ├── ADXDevice.cpp
│   │   ├── ADXQueue.cpp
│   │   ├── ADXCommandList.cpp
│   │   ├── ADXResource.cpp
│   │   ├── ADXHeap.cpp
│   │   ├── ADXDescriptorHeap.cpp
│   │   ├── ADXPipeline.cpp
│   │   └── ADXFence.cpp
│   │
│   ├── dxgi/
│   │   ├── ADXGIFactory.cpp
│   │   ├── ADXGIAdapter.cpp
```

```
│   │   ├── ADXGISwapChain.cpp
│   │   └── ADXGIFormat.cpp
│   │
│   ├── compiler/
│   │   ├── ADXIL/
│   │   ├── NIR/
│   │   └── ShaderCache/
│   │
│   ├── backend/
│   │   ├── metal/
│   │   └── vulkan_reference/
│   │
│   ├── platform/
│   │   ├── wine/
│   │   ├── macos/
│   │   └── win32abi/
│   │
│   └── dll/
│       ├── d3d12/
│       ├── d3d12core/
│       └── dxgi/
│
├── Shared/
│   ├── ADXCommon/
│   ├── WindowsABI/
│   ├── AppleGPUBridge/
│   ├── Logging/
│   └── Util/
│
├── tests/
│   ├── api/
│   ├── shader/
│   ├── memory/
│   ├── descriptors/
│   ├── synchronization/
│   ├── rendering/
│   └── differential/
│
├── external/
│   ├── core/
│   └── toolchain/
│
└── references/
    ├── refs.lock
    └── fetch-references.sh
```

---

24. Dependency Management Policy

24.1 Build Dependencies

Keep true build dependencies under:

```
external/core/
external/toolchain/
```

Recommended initial dependencies:

```
DirectX-Headers
dxil-spirv
metal-cpp
SPIRV-Cross
DirectXShaderCompiler
Mesa
```

24.2 Reference Repositories

Keep reference repositories out of the permanent build dependency graph.

Use:

```
references/
├── refs.lock
└── fetch-references.sh
```

Example:

```
vkd3d-proton          <commit>
vkd3d                 <commit>
dxvk                  <commit>
dxmt                  <commit>
D3D12TranslationLayer <commit>
D3D11On12             <commit>
OpenCLOn12            <commit>
DirectX-Specs         <commit>
```

AVK143 does not need to be mirrored as a third-party reference if it already lives in the developer's existing Khronos driver tree. The reference backend can point to or build against that project directly.

---

25. Suggested Initial Git Submodules

```
mkdir -p external/core external/toolchain references

git submodule add \
  https://github.com/microsoft/DirectX-Headers.git \
  external/core/DirectX-Headers

git submodule add \
  https://github.com/HansKristian-Work/dxil-spirv.git \
  external/core/dxil-spirv

git submodule add \
  https://github.com/apple/metal-cpp.git \
  external/core/metal-cpp

git submodule add \
  https://github.com/KhronosGroup/SPIRV-Cross.git \
  external/core/SPIRV-Cross

git submodule add \
  https://github.com/microsoft/DirectXShaderCompiler.git \
  external/toolchain/DirectXShaderCompiler

git submodule add \
  https://gitlab.freedesktop.org/mesa/mesa.git \
  external/core/mesa

git submodule update --init --recursive
```

Reference projects should normally be fetched separately and pinned.

---

26. Licensing Strategy

The project will interact with multiple license families.

Examples:

```
DirectX-Headers          MIT
D3D12TranslationLayer    MIT
D3D11On12                MIT
DirectX Graphics Samples MIT
D3D12MemoryAllocator     MIT
PixEvents                MIT
dxil-spirv               MIT
metal-cpp                Apache-2.0
DXVK                     zlib
vkd3d-proton             LGPL-2.1-or-later
DXMT current             LGPL-2.1-or-later
```

If ADX12 is intended to remain permissively licensed:

- prefer MIT/Zlib/Apache-compatible code for direct reuse;
- treat LGPL projects mainly as reference/oracle implementations unless deliberately complying with their terms;
- track every imported dependency in `THIRD_PARTY.md`;
- pin versions and commits;
- do not copy code across license boundaries without reviewing the consequences.

---

27. Bring-Up Roadmap

Phase 0 — Repository Bootstrap

Create:

```
README.md
LICENSE
THIRD_PARTY.md
docs/
ADX12/
Shared/
tests/
external/
references/
```

Set up:

- CMake or Meson;
- C++17/20;
- macOS build target;
- logging;
- unit-test harness;
- dependency bootstrap.

Milestone: repository builds an empty ADX12 shared library and test executable.

Phase 1 — ABI and COM Skeleton

Implement:

- DLL entry points;
- `D3D12CreateDevice`;
- COM lifetime management;
- `QueryInterface`;
- `AddRef`;
- `Release`;
- minimal `ID3D12Device`;
- adapter/capability placeholder;
- structured logging;
- HRESULT/error handling.

Milestone:

> A D3D12 application can load ADX12 and successfully obtain an `ID3D12Device`.

Phase 2 — Native Metal Device Bring-Up

Implement:

- Metal device discovery;
- `ADXMetalDevice`;
- queue creation;
- command-buffer creation;
- basic `MTLBuffer` allocation;
- basic `MTLTexture` allocation;
- minimal capability database.

Milestone:

> ADX12 creates a Metal device and submits a valid no-op command buffer.

Phase 3 — D3D12 Resources and Heaps

Implement:

- `ID3D12Heap`;
- `ID3D12Resource`;
- committed resources;
- initial placed-resource support;
- buffer creation;
- texture creation;
- mapping/upload/readback path;
- resource-state tracking skeleton.

Milestone:

> A D3D12 test creates buffers/textures, writes data, and reads expected data back.

Phase 4 — Command Lists and Queues

Implement:

- `ID3D12CommandAllocator`;
- `ID3D12GraphicsCommandList`;
- graphics queue;
- compute queue;
- copy queue;
- internal `ADXCommandStream`;
- first backend-neutral command opcodes.

Milestone:

> ADX12 records and executes copy and clear commands through Metal.

Phase 5 — Bootstrap Shader Pipeline

Enable:

```
DXIL
 ↓
dxil-spirv
 ↓
SPIR-V
 ↓
Mesa NIR
 ↓
MSL
 ↓
Metal
```

Implement:

- shader-module ingestion;
- shader-stage metadata;
- NIR lowering;
- MSL generation;
- Metal library creation;
- shader cache skeleton.

Milestone:

> A trivial compute shader executes correctly.

Phase 6 — Root Signatures and Descriptors

Implement:

- root signatures;
- root constants;
- root descriptors;
- CBV/SRV/UAV descriptors;
- sampler descriptors;
- descriptor heaps;
- descriptor tables;
- Metal argument-buffer mapping.

Milestone:

> A compute shader reads descriptors from a D3D12 descriptor heap and produces correct output.

Phase 7 — Graphics Pipeline State

Implement:

- graphics PSOs;
- vertex input;
- render-target state;
- depth/stencil state;
- raster state;
- blending;
- primitive topology;
- pipeline caching.

Milestone:

> First ADX12 triangle renders through the native Metal backend.

Phase 8 — DXGI and Presentation

Implement:

- `IDXGIFactory`;
- `IDXGIAdapter`;
- swapchain creation;
- format negotiation;
- `CAMetalLayer`;
- drawable acquisition;
- present;
- resize.

Milestone:

> A Windows D3D12 application displays a continuously presented image on macOS.

Phase 9 — Synchronization and Barriers

Implement:

- fences;
- queue signaling;
- queue waits;
- resource-state transitions;
- UAV barriers;
- aliasing barriers;
- initial enhanced-barrier model;
- Metal events/fences/barrier mapping.

Milestone:

> Multi-pass workloads produce correct results without implicit synchronization assumptions.

Phase 10 — AVK143 Reference Backend

Implement the ADX12 Vulkan reference backend:

```
ADX12
  ↓
ADX12VKBackend
  ↓
Vulkan
  ↓
AVK143
  ↓
Metal
```

Milestone:

> The same ADX12 tests can execute through both the native Metal backend and the AVK143 reference backend.

Phase 11 — Differential Validation Harness

Create:

```
tests/differential/
```

Compare:

```
ADX12 → Metal
ADX12 → AVK143 → Metal
vkd3d-proton → AVK143 → Metal
```

Record:

- image hashes;
- buffer hashes;
- query values;
- reported features;
- error codes;
- synchronization results.

Milestone:

> Basic rendering, compute, memory, and descriptor tests produce matching results across reference paths.

Phase 12 — Native ADXIL Frontend

Begin replacement of the bootstrap compiler.

Implement:

- DXIL container parser;
- DXIL metadata parser;
- LLVM/DXIL decoder;
- Shader Model semantic handling;
- ADXIL IR;
- ADXIL → NIR lowering.

Milestone:

```
DXIL → ADXIL → NIR → MSL
```

works for the initial shader corpus without `dxil-spirv`.

Phase 13 — Advanced D3D12 Features

After the baseline is stable:

- enhanced barriers;
- `ExecuteIndirect`;
- mesh shaders;
- variable-rate shading;
- sampler feedback;
- advanced residency;
- pipeline libraries;
- shader cache persistence;
- Shader Model 6.x expansion;
- DXR;
- HDR;
- video features where feasible.

These should be implemented only after the basic runtime has strong tests.

---

28. Initial Minimum Viable ADX12 Stack

```
ADX12 v0.0
│
├── DirectX-Headers
├── COM / D3D12 ABI skeleton
├── D3D12CreateDevice
├── ID3D12Device
├── ID3D12CommandQueue
├── ID3D12CommandAllocator
├── ID3D12GraphicsCommandList
├── ID3D12Resource
├── ID3D12Heap
├── descriptor heaps
├── root signatures
├── graphics + compute PSOs
├── fences
│
├── DXIL
│     ↓
│   dxil-spirv
│     ↓
│   SPIR-V
│     ↓
│   Mesa NIR
│     ↓
│   MSL
│
├── Metal backend
│
└── DXGI
      ↓
    CAMetalLayer
```

The first goal is not broad feature coverage.

The first goal is a complete vertical path:

```
Windows application
        │
        ▼
D3D12 ABI
        │
        ▼
ADX12 runtime
        │
        ▼
shader compiler
        │
```

```
       ▼
Metal backend
       │
       ▼
CAMetalLayer / AGX
```

---

29. First Test Corpus

The initial test set should be tiny and deterministic.

API tests

```
create_device
query_interface
device_feature_query
create_queue
create_command_allocator
create_command_list
create_heap
create_resource
create_descriptor_heap
create_fence
```

Memory tests

```
upload_buffer
readback_buffer
copy_buffer
placed_resource
resource_alias
```

Shader tests

```
compute_constant
compute_buffer
vertex_passthrough
pixel_constant
descriptor_read
sampler_read
```

Rendering tests

```
clear_rtv
triangle
indexed_triangle
textured_triangle
depth_test
blend_test
```

Synchronization tests

```
queue_fence
cross_queue_wait
resource_transition
uav_barrier
```

---

30. Feature Tracking

Create:

```
docs/FeatureMatrix.md
```

Example structure:

Area	Feature	ADX12	Metal Backend	AVK143 Ref	Test
Device	D3D12CreateDevice	WIP	N/A	N/A	Yes
Resource	Buffers	WIP	WIP	Planned	Yes
Resource	Textures	Planned	Planned	Planned	Planned
Descriptors	CBV/SRV/UAV	Planned	Planned	Planned	Planned
Sync	Fences	Planned	Planned	Planned	Planned
Graphics	Basic PSO	Planned	Planned	Planned	Planned
Shader	DXIL bootstrap	Planned	Planned	N/A	Planned
DXGI	Swapchain	Planned	Planned	N/A	Planned

This prevents project status from becoming dependent on memory, screenshots, or enthusiastic commit messages.

---

31. Documentation Set

The repository should gradually grow the following documents:

```
docs/
├── ADX12Architecture.md
├── ABI.md
├── COMModel.md
├── D3D12MetalMapping.md
├── DXGIModel.md
├── ResourceModel.md
├── MemoryModel.md
├── DescriptorModel.md
├── RootSignatureModel.md
├── CommandModel.md
├── Synchronization.md
├── DXILNIRMapping.md
├── ADXIL.md
├── MetalBackend.md
├── AVK143ReferenceBackend.md
├── ValidationStrategy.md
├── FeatureMatrix.md
├── CompatibilityNotes.md
└── REFERENCES.md
```

---

32. Development Rules

Rule 1

Do not add a backend-specific object to the public runtime unless necessary.

Bad:

```
ID3D12Resource → VkImage
```

Preferred:

```
ID3D12Resource → ADXResource → backend resource
```

Rule 2

Every D3D12-visible semantic decision should cite:

```
Microsoft specification
reference implementation behavior
ADX12 test
```

Rule 3

Keep the native Metal backend independent from the AVK143 reference backend.

Rule 4

Use AVK143 for validation, not as a permanent hidden dependency of the native Metal path.

Rule 5

Do not implement advanced features until the lower layers have deterministic tests.

Rule 6

Keep compiler bring-up and runtime bring-up separable.

Rule 7

Maintain a clean licensing boundary around LGPL reference implementations.

---

33. Immediate Bootstrap Checklist

- ☐ Create `Microsoft_AppleDrivers` repository.
- ☐ Add `README.md`.
- ☐ Choose project license.
- ☐ Add `THIRD_PARTY.md`.
- ☐ Create `docs/`.
- ☐ Create `ADX12/`.
- ☐ Create `Shared/`.
- ☐ Create `tests/`.
- ☐ Create `external/core/`.
- ☐ Create `external/toolchain/`.
- ☐ Create `references/`.
- ☐ Add DirectX-Headers.
- ☐ Add metal-cpp.
- ☐ Add dxil-spirv.
- ☐ Add Mesa.
- ☐ Add DXC.
- ☐ Add SPIRV-Cross.
- ☐ Pin vkd3d-proton as the primary semantic reference.
- ☐ Pin upstream VKD3D.
- ☐ Pin DXMT.
- ☐ Pin DXVK.
- ☐ Pin D3D12TranslationLayer.
- ☐ Pin D3D11On12.
- ☐ Pin OpenCLOn12.
- ☐ Pin DirectX-Specs.
- ☐ Define how ADX12 locates/builds against AVK143 for reference tests.
- ☐ Implement build system.
- ☐ Implement logging.
- ☐ Implement basic COM utilities.
- ☐ Implement `D3D12CreateDevice`.
- ☐ Bring up native Metal device creation.
- ☐ Start deterministic unit tests.

---

34. Architectural Identity

ADX12 should ultimately be understood as:

```
A Direct3D 12 userspace implementation
        +
A D3D12 semantic runtime
        +
A DXIL/NIR compiler stack
        +
A native Metal backend
        +
An AVK143-based validation backend
        +
A Wine/CrossOver integration layer
```

It should **not** be described merely as:

```
D3D12 → Metal translator
```

The distinction is important.

The intended implementation owns the full graphics-runtime path below the Windows-facing ABI.

---

35. Relationship to the Existing Driver Family

Conceptually:

```
Khronos_AppleICDs
│
├── AO46
│   OpenGL
│      ↓
│   Mesa / NIR
│      ↓
│   Apple backend
│
└── AVK143
    Vulkan
       ↓
    KosmicKrisp / NIR
       ↓
    Metal


Microsoft_AppleDrivers
│
└── ADX12
    Direct3D 12
       ↓
    ADX12 Runtime
       ↓
    ADXIL / NIR
       ↓
    Metal
```

The projects can share knowledge and carefully selected infrastructure while keeping their public API semantics isolated.

The resulting driver family becomes:

```
OpenGL 4.6  ── AO46  ──┐
                       │
Vulkan 1.4  ── AVK143 ─┼──► Apple GPU / Metal infrastructure
                       │
Direct3D 12 – ADX12  ──┘
```

---

36. Final Starting Priority

The recommended order for gathering external material and beginning implementation is:

1. **DirectX-Headers**
   Establish the real ABI and public interfaces.
2. **vkd3d-proton**
   Use as the primary D3D12 semantic and real-world behavior reference.
3. **D3D12TranslationLayer**
   Study Microsoft's resource, descriptor, batching, and runtime design.
4. **DXC + dxil-spirv**
   Establish the first usable DXIL pipeline.
5. **Mesa / NIR + existing KosmicKrisp knowledge**
   Establish the common shader compiler layer.
6. **metal-cpp**
   Build the native Metal backend.
7. **DXMT + DXVK**
   Study Wine/macOS integration, DXGI, COM, DLL deployment, caches, and runtime design.
8. **AVK143**
   Use as the project-owned Vulkan-over-Metal reference backend for differential validation.
9. **DirectX Graphics Samples + custom tests**
   Build deterministic bring-up workloads.

---

37. Initial Success Definition

The first meaningful ADX12 milestone is achieved when a Windows D3D12 test program running under Wine/CrossOver can:

1. load ADX12;
2. call `D3D12CreateDevice`;
3. create a command queue;
4. create a resource;
5. compile a trivial DXIL shader through the bootstrap path;
6. create a graphics or compute pipeline;
7. submit work through the native Metal backend;
8. receive correct output;
9. present through DXGI/CAMetalLayer where applicable;
10. reproduce the same expected result through the AVK143 reference path.

At that point ADX12 has moved beyond architecture sketches and become a functioning D3D12 implementation skeleton.

Summary

ADX12 begins with one central principle:

> **Preserve D3D12 semantics at the frontend, own the runtime in the middle, and make Metal a true backend.**

vkd3d-proton provides the strongest external semantic reference. Microsoft open-source projects provide ABI and runtime design knowledge. Mesa/NIR provides the compiler foundation. `dxil-spirv` provides the bootstrap shader path. `metal-cpp` provides the native Metal interface. DXMT and DXVK provide valuable Wine/platform design references.

AVK143 replaces MoltenVK in the project architecture and becomes the **project-owned Vulkan-over-Metal differential validation backend**.

The desired final path is:

```
D3D12 application
      ↓
Wine / CrossOver
      ↓
ADX12
      ↓
ADX12 Runtime
      ↓
ADXIL / NIR
      ↓
Metal Backend
      ↓
Metal
```

```
      ↓
AGX
```

with the validation path:

```
D3D12 workload
      ↓
vkd3d-proton / ADX12VKBackend
      ↓
Vulkan
      ↓
AVK143
      ↓
Metal
      ↓
AGX
```

That gives the project both a native implementation path and a controlled reference path from the beginning.

38. Verified Upstream Reference Facts

The following statements were re-checked against upstream repositories on 23 August 2026 and should be treated as **reference facts**, not ADX12 implementation claims.

DirectX-Headers

Microsoft's `DirectX-Headers` repository hosts the official Direct3D 12 headers and makes them available under the MIT license. The repository includes core D3D12 headers, helpers such as `d3dx12`, GUID support, IDL files, and build integration for CMake and Meson.

Reference: https://github.com/microsoft/DirectX-Headers

vkd3d-proton

vkd3d-proton describes itself as a fork of VKD3D that aims to implement the full Direct3D 12 API on top of Vulkan and serves as the D3D12 development effort for Proton. It is intended to be used as native Win32 `d3d12.dll` and `d3d12core.dll` replacements and shares DXGI with DXVK. Its design aggressively uses modern Vulkan capabilities for performance and compatibility.

Reference: https://github.com/HansKristian-Work/vkd3d-proton

dxil-spirv

`dxil-spirv` translates Shader Model 6.x DXIL to SPIR-V for D3D12 translation libraries and maintains a reference shader test suite. Its repository currently identifies the project as MIT licensed.

Reference: https://github.com/HansKristian-Work/dxil-spirv

D3D12TranslationLayer

Microsoft's `D3D12TranslationLayer` is a helper library for translating higher-level/traditional graphics constructs into D3D12-style execution. Its documented responsibilities include resource binding, renaming, suballocation and deferred destruction, batching/threading, PSO compilation, and residency management.

Reference: https://github.com/microsoft/D3D12TranslationLayer

D3D11On12

Microsoft's `D3D11On12` repository explicitly describes the project as an implementation of the D3D11 **user-mode DDI**, not as a replacement `d3d11.dll`. It is therefore particularly useful as an example of a driver-like layer adapting one graphics contract to another runtime.

Reference: https://github.com/microsoft/D3D11On12

OpenCLOn12

Microsoft's `OpenCLOn12` repository documents a dependency on Mesa compiler infrastructure for consuming OpenCL C/SPIR-V and producing DXIL. This is a useful precedent for combining Microsoft API work with Mesa compiler components.

Reference: https://github.com/microsoft/OpenCLOn12

DirectX-Specs

`DirectX-Specs` publishes engineering specifications for numerous D3D12 features, including enhanced barriers, feature level 12_2, mesh shaders, ray tracing, sampler feedback, variable rate shading, work graphs, resource binding, indirect drawing, shader models, and other advanced areas. Microsoft notes that these documents supplement official API documentation and that some older material may not reflect later changes.

Reference: https://github.com/microsoft/DirectX-Specs

metal-cpp

Apple's `metal-cpp` repository describes the project as a low-overhead, header-only C++ interface for Metal, requiring C++17 and providing direct C++ mappings for Metal classes, enums, and constants. The repository is distributed under Apache-2.0.

Reference: https://github.com/apple/metal-cpp

39. ADX12 Internal Layer Contract

A recurring implementation risk is allowing backend details to leak upward until the runtime becomes impossible to reason about. The following contract should therefore be enforced.

Frontend layer responsibilities

The frontend owns:

- exported D3D12/DXGI entry points;
- COM identity and lifetime rules;
- argument validation that ADX12 chooses to perform;
- application-visible feature reporting;
- translation of public structures into ADX12 internal structures;
- stable ABI-facing error codes and behavior.

The frontend must not own:

- raw Metal object lifetime;
- backend queue encoding;
- direct shader source generation;
- backend-specific synchronization policy.

Runtime layer responsibilities

The runtime owns:

- ADX12 devices and adapters;
- resource identity and lifetime;
- heaps and allocation policy;
- descriptors and root-layout semantics;
- command-list state;
- queue semantics;
- resource-state tracking;
- barrier analysis;
- fence/query behavior;
- pipeline object identity and caches;
- backend-neutral command representation.

Compiler layer responsibilities

The compiler owns:

- DXIL ingestion;
- bootstrap conversion through `dxil-spirv` where needed;
- long-term ADXIL parsing/lowering;
- NIR optimization and lowering;

- resource-binding metadata;
- shader feature analysis;
- MSL emission interfaces;
- shader cache keys and compiler-version compatibility.

Backend layer responsibilities

The native Metal backend owns:

- `MTLDevice` discovery;
- command queues and command buffers;
- render/compute/blit encoders;
- MTLBuffer/MTLTexture creation;
- argument-buffer/resource-binding realization;
- Metal pipeline state creation;
- Metal fences/events/barriers;
- `CAMetalLayer` and drawable integration where delegated by DXGI;
- GPU-specific capability mapping.

The AVK143 reference backend owns only the alternate Vulkan execution path needed for testing and comparison.

40. Error Model and Diagnostics

ADX12 should implement diagnostics from the beginning rather than adding them after compatibility failures become impossible to localize.

Recommended logging domains:

```
ADX12_ABI
ADX12_COM
ADX12_DEVICE
ADX12_RESOURCE
ADX12_MEMORY
ADX12_DESCRIPTOR
ADX12_COMMAND
ADX12_SYNC
ADX12_PIPELINE
ADX12_SHADER
ADX12_DXGI
ADX12_METAL
ADX12_AVK143_REF
ADX12_VALIDATION
```

Recommended severity levels:

```
TRACE
DEBUG
INFO
WARN
ERROR
FATAL
```

Every object should have a stable internal debug ID. Important logs should include:

- object type and ID;
- D3D12-visible parameters;
- relevant ADX12 internal state;
- backend object identifier when safe;
- thread ID;
- queue ID;
- frame/submission sequence number;
- HRESULT returned to the application.

For differential tests, logs from the Metal and AVK143 paths should use comparable event names so traces can be aligned mechanically.

41. Performance Design Principles

Correctness comes first during bring-up, but some architectural decisions strongly affect whether the project can ever become performant.

Avoid one-call-one-call translation

D3D12 command recording should feed an internal command/state model that can coalesce redundant changes and encode Metal work efficiently.

Cache expensive objects

At minimum cache:

- root-layout translations;
- Metal argument layouts;
- shader compilation products;
- graphics and compute pipeline states;
- format/conversion decisions;
- immutable sampler state;
- frequently used null descriptors/resources.

Keep application-visible synchronization exact, backend synchronization minimal

D3D12 synchronization semantics must be preserved, but the backend should avoid emitting heavyweight Metal synchronization where dependency analysis proves it unnecessary.

Exploit unified memory without breaking D3D12 semantics

Apple Silicon's unified memory can reduce some copies and allocation complexity, but ADX12 must still preserve the visibility, lifetime, and ordering behavior expected by D3D12 applications.

Separate correctness fallbacks from fast paths

Each difficult feature should be allowed to have:

```
correct baseline path
        +
optimized Metal-specific path
```

Tests must ensure both produce identical externally observable results.

42. Risk Register

Risk	Impact	Mitigation
D3D12 and Metal synchronization mismatch	Incorrect rendering, hangs, data hazards	Maintain explicit ADX barrier/resource-state model; compare against vkd3d-proton + AVK143
Descriptor model mismatch	Corruption or shader-visible wrong resources	Keep ADX descriptor IR; validate root-signature cases systematically
DXIL semantic gaps	Shader failures across modern games	Bootstrap via dxil-spirv, maintain shader corpus, build ADXIL incrementally
Metal feature gaps versus D3D12 requirements	Unsupported features or expensive emulation	Feature matrix; truthful capability reporting; isolate emulation paths
COM/ABI incompatibility	Applications fail before GPU work begins	Use official DirectX-Headers; build dedicated ABI/COM tests
Wine/CrossOver integration differences	DLL load, window, or DXGI failures	Keep platform layer separate; test minimal native DLL loading first
Pipeline compilation stutter	Poor game experience	Persistent caches, asynchronous compilation where semantics allow
License contamination	Distribution constraints	Track SPDX/license boundaries and keep LGPL references isolated
Reference backend hides shared bug	False confidence	Use multiple oracles: spec, vkd3d-proton, AVK143, targeted expected-output tests
Scope explosion	Project stalls before first triangle	Enforce phased roadmap and MVP gates
Trademark/name confusion	Users infer Microsoft sponsorship	Prominent independence notice; consider a neutral public repository name

43. Contribution and Review Rules

A contribution should not be accepted merely because one game appears to render correctly.

Each functional change should, where practical, include:

1. the D3D12 behavior being implemented;
2. the relevant Microsoft specification or header/interface reference;
3. a focused ADX12 test;
4. expected behavior under the native Metal backend;
5. comparison against the AVK143 or vkd3d-proton reference path when relevant;
6. any feature-reporting change;
7. licensing provenance for imported code.

Backend work should be rejected if it silently changes public D3D12 semantics to fit Metal more conveniently.

Compatibility workarounds should be documented separately from spec-correct behavior and should include the application or pattern that requires the workaround.

44. Suggested Issue Taxonomy

Recommended issue labels:

```
area:abi
area:com
area:device
area:resource
area:memory
area:descriptor
area:command
area:sync
area:pipeline
area:dxil
area:adxil
area:nir
area:metal
area:dxgi
area:wine
area:avk143-ref
area:validation
area:performance
area:compat
area:docs

kind:bug
kind:feature
kind:conformance
kind:refactor
kind:research
kind:performance
kind:test

priority:p0
priority:p1
priority:p2
priority:p3
```

Milestone labels can follow the bring-up phases in this document.

45. Long-Term Expansion Map

Once the baseline runtime is stable, advanced work can branch into several areas.

Advanced graphics

- enhanced barriers;
- mesh shaders;
- variable rate shading where Metal capabilities allow meaningful mapping;
- sampler feedback or emulation strategy;
- work graphs or future compute-graph models where feasible;
- advanced indirect execution;
- pipeline libraries and persistent shader/pipeline caches.

Ray tracing

DXR should be treated as a dedicated project phase rather than folded casually into baseline rendering. The design should separate:

```
D3D12 state objects / DXR semantics
        -> ADX12 ray-tracing IR
        -> Metal ray-tracing backend
```

and validate each supported semantic area independently.

Tooling

Future developer tooling could include:

- ADX12 object/state dumps;
- command stream disassembler;
- descriptor heap inspector;
- NIR/MSL shader dump modes;
- Metal capture integration;
- differential image/buffer comparator;
- compatibility database;
- feature-capability report generator.

Appendix A - Bootstrap Dependency Classification

Resource	Role	Build dependency?	Primary reason
DirectX-Headers	ABI	Yes	Official D3D12 headers/IDL/GUIDs
DXC	Toolchain	Yes/Tool	Produce and inspect DXIL
dxil-spirv	Compiler bootstrap	Yes initially	DXIL -> SPIR-V
Mesa	Compiler infrastructure	Yes	NIR and SPIR-V ingestion
metal-cpp	Native backend interface	Yes	C++ Metal API
SPIRV-Cross	Compiler/reference utility	Optional/Yes initially	MSL/reference comparison
SPIRV-Tools	Validation utility	Optional/Tool	SPIR-V validation/disassembly
Wine	Platform/tooling	External	Win32 environment and `widl`
vkd3d-proton	Semantic oracle	No	Real-world D3D12 implementation reference
upstream VKD3D	Semantic oracle	No	General-purpose D3D12 reference
DXMT	macOS/Wine/Metal reference	No	D3D-to-Metal integration patterns
DXVK	DXGI/Wine reference	No	DXGI, COM, DLL, cache architecture
D3D12TranslationLayer	Microsoft architecture reference	No	Resources, descriptors, batching, residency
D3D11On12	Microsoft DDI reference	No	User-mode driver-style design
OpenCLOn12	Mesa/Microsoft compiler reference	No	Mesa compiler integration precedent
DirectX-Specs	Specification reference	No	Advanced D3D12 engineering specs
AVK143	Project-owned validation backend	Project dependency for ref tests	Vulkan -> Metal differential path

Appendix B - Core Architecture Diagrams

B.1 Shader bootstrap and long-term compiler path

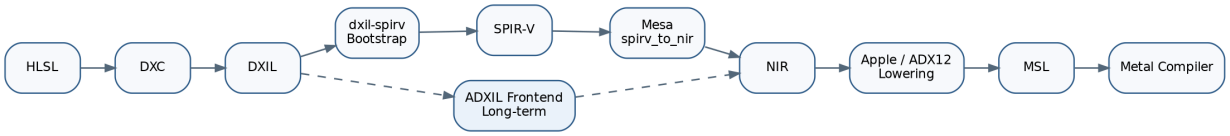


Figure 1: ADX12 shader compiler strategy

B.2 Resource abstraction

Figure 2: ADX12 resource abstraction

B.3 Differential validation topology

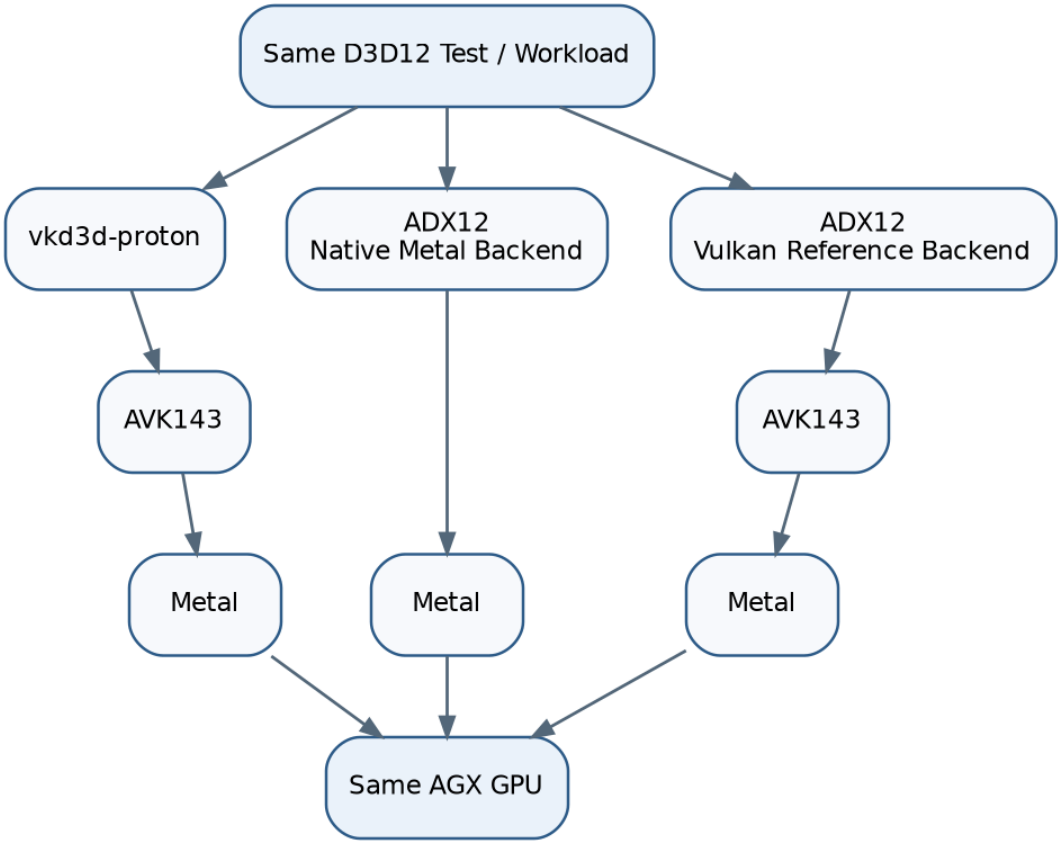


Figure 3: ADX12 differential validation topology

Appendix C - Minimal Differential Test Record

Each differential test should emit a compact record similar to:

```
Test: descriptor_table_cbv_001
D3D12 feature: CBV descriptor table
Expected output hash: <known>

ADX12 Metal:
  result: PASS
  image/buffer hash: <hash>
  warnings: none

ADX12 AVK143 reference:
  result: PASS
  image/buffer hash: <hash>
  warnings: none

vkd3d-proton + AVK143:
  result: PASS
  image/buffer hash: <hash>

Conclusion:
  semantic agreement: YES
```

Appendix D - Documentation Maintenance Rules

This document should evolve with implementation rather than becoming a frozen proposal.

Update it whenever any of the following changes:

- public project scope;
- runtime/backend boundary;
- shader compiler architecture;
- AVK143 reference strategy;
- dependency or licensing policy;
- major feature-reporting policy;
- repository structure;
- release/bring-up milestones.

The `FeatureMatrix.md` file should remain the authoritative implementation-status summary, while this PDF remains the architecture and project-history reference.

Reference Index

1. Microsoft DirectX-Headers - https://github.com/microsoft/DirectX-Headers
2. vkd3d-proton - https://github.com/HansKristian-Work/vkd3d-proton
3. upstream VKD3D - https://gitlab.winehq.org/wine/vkd3d
4. dxil-spirv - https://github.com/HansKristian-Work/dxil-spirv
5. Microsoft DirectX Shader Compiler - https://github.com/microsoft/DirectXShaderCompiler
6. Mesa - https://gitlab.freedesktop.org/mesa/mesa
7. Apple metal-cpp - https://github.com/apple/metal-cpp
8. SPIRV-Cross - https://github.com/KhronosGroup/SPIRV-Cross
9. SPIRV-Tools - https://github.com/KhronosGroup/SPIRV-Tools
10. Wine - https://gitlab.winehq.org/wine/wine
11. Microsoft D3D12TranslationLayer - https://github.com/microsoft/D3D12TranslationLayer
12. Microsoft D3D11On12 - https://github.com/microsoft/D3D11On12
13. Microsoft OpenCLOn12 - https://github.com/microsoft/OpenCLOn12
14. Microsoft DirectX-Specs - https://github.com/microsoft/DirectX-Specs
15. Microsoft DirectX Graphics Samples - https://github.com/microsoft/DirectX-Graphics-Samples
16. D3D12 Memory Allocator - https://github.com/GPUOpen-LibrariesAndSDKs/D3D12MemoryAllocator
17. Microsoft PixEvents - https://github.com/microsoft/PixEvents
18. DXMT - https://github.com/3Shain/dxmt
19. DXVK - https://github.com/doitsujin/dxvk
20. RenderDoc - https://github.com/baldurk/renderdoc

End of architecture specification.